1. Matematický model (Stavový prostor)

Pro tepelné systémy se nejčastěji používá model RC (Resistor-Capacitor), který je analogií k elektrickým obvodům. Základní diferenciální rovnice pro změnu vnitřní teploty vypadá takto:

$$C\frac{dT_{in}}{dt} = \frac{T_{out} - T_{in}}{R} + P_h \cdot \eta + Q_{sun} - Q_{loss}$$

Kde:

- T_{in}, T_{out}: Vnitřní a venkovní teplota.
- C: Tepelná kapacita objektu (setrvačnost).
- R: Tepelný odpor obálky budovy.
- P_h: Okamžitý výkon zdroje.
- η: Účinnost.
- Q_{sun}, Q_{loss}: Zisky ze slunce (oblačnost) a ztráty (např. otevřené okno).

V diskrétním čase pro stavový prostor (vhodné pro Python) můžeme model zapsat jako:

$$x_{k+1} = Ax_k + Bu_k + Ed_k$$

- x_k: Vnitřní teplota v čase k.
- u_k: Výkon zdroje (akční zásah).
- d_k: Vektor poruch (venkovní teplota, oblačnost, otevřené okno).

2. Návrh řešení v Pythonu

Pro tento účel využijeme knihovnu `**scipy.optimize**` pro minimalizaci nákladů a lineární regresi pro odhad parametrů haly z vašich historických dat.

```python
import numpy as np
from scipy.optimize import minimize

class ThermalModelMPC:
  def __init__(self, R=0.1, C=5000):
```

```
        # Počáteční odhad parametrů (budou zpřesněny z dat)
        self.R = R  # Tepelný odpor
        self.C = C  # Tepelná kapacita (setrvačnost)

    def predict_next_temp(self, T_in, T_out, P_heat, window_open, dt=3600):
        """Predikce teploty v dalším kroku (1 hodina)"""
        # Pokud je okno otevřené, odpor R se drasticky sníží
        effective_R = self.R / 5 if window_open else self.R

        dT = ( (T_out - T_in) / effective_R + P_heat ) * (dt / self.C)
        return T_in + dT

    def objective_function(self, u, *args):
        """Cílová funkce: Minimalizace spotřeby při dodržení komfortu"""
        T_start, T_out_forecast, cloud_forecast, schedule, T_setpoint = args
        cost = 0
        T_current = T_start

        for i in range(len(u)):
            # Výkon zdroje v čase i
            P_h = u[i]

            # Predikce budoucí teploty
            T_next = self.predict_next_temp(T_current, T_out_forecast[i], P_h, False)

            # Penalizace za spotřebu energie
            cost += P_h * 1.0

            # Penalizace za nedodržení teploty (pouze pokud je provozní doba)
            if schedule[i] == 1:
                if T_next < T_setpoint:
                    cost += (T_setpoint - T_next) * 5000  # Vysoká penalizace za chlad

            T_current = T_next
        return cost

# --- Příklad použití ---
# Předpověď na 12 hodin (venkovní teplota)
T_out_12h = [2, 1, 0, -1, -1, 0, 2, 4, 6, 7, 8, 8]
# Provozní program (1 = v provozu, 0 = útlum) - např. start v 06:00
work_schedule = [0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1]

model = ThermalModelMPC()
initial_power_guess = np.zeros(12) # Startovní odhad výkonu

# Hledání optimálního plánu výkonu
res = minimize(
```

```
  model.objective_function,
  initial_power_guess,
  args=(20.0, T_out_12h, None, work_schedule, 21.0),
  bounds=[(0, 100)] * 12 # Výkon zdroje 0-100%
)

optimal_plan = res.x
print(f"Navržený plán výkonu na 12h: {optimal_plan}")
```

3. Jak využít historická data (Identifikace systému)

Aby model fungoval, musíte z tabulky dat zjistit hodnoty **R** a **C**. Toho docílíte **regresní analýzou**:

1. Z tabulky vypočítejte $\Delta T = T_{in}(k+1) - T_{in}(k)$.
2. Sestavte soustavu rovnic, kde nezávislé proměnné jsou $(T_{out} - T_{in})$ a P_{heat}.
3. Koeficienty regrese vám přímo dají hodnoty $1/RC$ a $1/C$.

4. Strategie optimalizace

Vzhledem k zadání doporučuji tyto principy:

Pre-heating (Předehřev): Model díky setrvačnosti zjistí, že musí zapnout VZT už např. ve 4:00 ráno na 30% výkon, aby v 6:00 byla teplota přesně na 21 °C, místo aby zapnul na 100% v 6:00 (což bývá neefektivní).
Využití oblačnosti: Pokud předpověď hlásí jasno, model může snížit výkon zdroje dříve, protože počítá se solárními zisky okny/pláštěm.
Detekce anomálií (Okno): Pokud reálná teplota klesá rychleji, než predikuje model, systém identifikuje "poruchovou veličinu" (otevřené okno) a okamžitě přizpůsobí plán – buď zvýší výkon, nebo (pokud je to ekonomičtější) zdroj vypne, aby netopil "do ulice".

Tento model je **adaptivní**. S každým dalším dnem provozu se parametry **R** a **C** zpřesňují, čímž se minimalizuje odchylka mezi predikcí a realitou.

5. Vysvětlení matematického modelu a stavového prostoru

A. První rovnice: Tepelná bilance (Fyzikální pohled)

Představte si halu jako **obrovský sud s vodou**, který má ale ve stěnách malé dírky.

$$C \cdot \frac{dT_{in}}{dt} = \frac{T_{out} - T_{in}}{R} + P_{heat} \cdot \eta + Q_{zisky}$$

- **Levá strana ($C \cdot \frac{dT_{in}}{dt}$):** To je "výsledek".
  - $\frac{dT_{in}}{dt}$ říká, jak rychle se mění teplota (stoupá nebo klesá).
  - C je **tepelná setrvačnost**. Čím je hala větší a čím víc je v ní betonu a strojů, tím je C větší. Je to jako setrvačník – čím je těžší, tím hůře se rozbíhá, ale pak dlouho jede sám.
- **Člen $\frac{T_{out}-T_{in}}{R}$:** To jsou **ztráty**.
  - Rozdíl teplot ($T_{out} - T_{in}$) tlačí teplo ven (nebo dovnitř).
  - R je **izolace**. Čím větší je R, tím méně tepla uteče. Je to jako odpor v elektrice nebo průměr té "dírky v sudu".
- **Člen $P_{heat} \cdot \eta$:** To je tvůj **radiátor/VZT**.
  - P_{heat} je výkon, který platíš, a η (účinnost) říká, kolik z toho se skutečně promění v teplo uvnitř.
- **Člen Q_{zisky}:** To jsou **bonusy**.
  - Slunce svítící do oken, běžící stroje nebo lidé v hale. Ti všichni halu ohřívají "zadarmo".

Lidsky řečeno: Změna teploty v hale závisí na tom, kolik tepla tam dodáš (topení + slunce) minus to, co ti uteče špatnou izolací.

B. Druhá rovnice: Stavový prostor (Pohled pro počítač)

Počítač neumí počítat s plynulým časem (derivacemi), on vidí svět v "krocích" (např. každou hodinu). Proto rovnici přepisujeme na "recept", jak vypočítat budoucnost z přítomnosti.

$$x_{k+1} = A \cdot x_k + B \cdot u_k + E \cdot d_k$$

- x_{k+1}: Teplota v **příští hodině** (budoucnost).
- x_k: Teplota **teď** (přítomnost).
- A: Koeficient "vychladnutí". Říká nám, kolik procent tepla si hala udrží z minulé hodiny jen díky své izolaci a setrvačnosti.
- u_k: **Tvůj zásah** (výkon topení). Koeficient B pak určuje, o kolik stupňů tenhle výkon halu ohřeje za jednu hodinu.
- d_k: **Vnější vlivy (poruchy)**. To je ta předpověď počasí nebo otevřené okno. Koeficient E říká, jak moc tyto vlivy s teplotou zamávají.

Proč se to zapisuje takto?

Protože pro Python (a matici) je to extrémně rychlé. Když chceš vědět, co se stane za 12 hodin, stačí tuhle rovnici spustit 12x za sebou v cyklu:

1. Vezmeš dnešní teplotu $\rightarrow$ spočítáš tu za hodinu.
2. Vezmeš tu za hodinu $\rightarrow$ spočítáš tu za dvě.
3. A tak dále.

Kouzlo optimalizace:
Jelikož máš tuhle rovnici, můžeš počítači říct: *"Zkus tisíc různých kombinací výkonu topení (u) a najdi tu, kde bude spotřeba nejnižší, ale v čase $k+6$ (začátek směny) bude teplota přesně tam, kde ji chci."* To je princip toho, co jsem ti navrhl v Python kódu.

6. Vztah mezi fyzikálními parametry C, R a konstantami A, B, E

Tyto dva světy jsou přímo propojeny. Přechod od fyzikálních parametrů (**C**, **R**) k těm "počítačovým" (**A**, **B**, **E**) se provádí pomocí procesu, kterému se říká **diskretizace**.

Představ si, že tvůj počítač měří teplotu v pravidelných intervalech **Delta t** (např. každou hodinu, tedy 3600 sekund).

Odvození vztahu

Pokud vyjdeme ze zjednodušené fyzikální rovnice (pro přehlednost teď vynecháme sluneční zisky):

$$C\frac{dT_{in}}{dt} = \frac{T_{out} - T_{in}}{R} + P_{heat}$$

Po matematické úpravě (náhradě derivace diferencí) získáme vztahy pro koeficienty v diskrétním čase:

Koeficient A (Setrvačnost systému)
Určuje, kolik tepla v hale zůstane "samo od sebe" z předchozího kroku.

$$A = 1 - \frac{\Delta t}{R \cdot C}$$

Logická úvaha: Pokud by byla izolace **R** nekonečná (dokonalá), **A** by se rovnalo 1. To znamená, že teplota v další hodině bude přesně stejná jako v té předchozí. Čím menší je odpor **R** nebo kapacita **C**, tím víc **A** klesá pod 1 (hala rychleji chladne).

Koeficient B (Účinnost topení)
Určuje, jak moc pohne s teplotou zdroj tepla.

$$B = \frac{\Delta t}{C}$$

Logická úvaha: Tady je vidět, že odpor izolace nehraje roli. Důležitá je jen tepelná kapacita **C**. Čím je hala "těžší" (větší **C**), tím menší vliv má stejná dávka energie na změnu teploty.

Koeficient E (Vliv okolí)
Určuje, jak moc nás ovlivňuje venkovní teplota.

$$E = \frac{\Delta t}{R \cdot C}$$

*Logická úvaha: Součet **A** + **E** = **1**. To dává smysl: budoucí teplota je v podstatě vážený průměr mezi aktuální vnitřní teplotou a venkovní teplotou, posunutý o to, kolik jsme si přitopili.

7. Souhrnná tabulka pro regresní model

Pokud chceš tyto vztahy zapsat do technické dokumentace, vypadá to takto:

Koeficient	Fyzikální význam	Vztah k R a C
A	Schopnost udržet si teplotu	$1 - \frac{\Delta t}{RC}$
B	Citlivost na výkon zdroje	$\frac{\Delta t}{C}$
E	Citlivost na venkovní mráz	$\frac{\Delta t}{RC}$

Exportovat do Tabulek

Proč je to pro tebe důležité?

Když máš tu tabulku historických dat, o které jsi mluvil, tvůj algoritmus (např. lineární regrese) nemusí složitě hádat R a C. On prostě najde nejlepší čísla pro A, B a E, která odpovídají realitě.

Z nich pak můžeš zpětně vypočítat:

- **Tepelnou kapacitu haly:** $C = \frac{\Delta t}{B}$
- **Tepelný odpor (izolaci):** $R = \frac{B}{E}$

Tímto způsobem ti data z provozu prozradí, jak na tom tvá budova skutečně je, aniž bys musel lézt na střechu a měřit tloušťku vaty.

8. Výpočet konstant A, B a E z historických dat pro první iteraci

Pokud si to chceš spočítat ručně (například v Excelu nebo na papíře), nepotřebuješ hlubokou znalost statistiky. Stačí ti k tomu metoda, které se říká **metoda nejmenších čtverců**, ale pro začátek to zkusíme intuitivně přes „diference“ (rozdíly).

Máš rovnici:

$$T_{in}(k+1) = A \cdot T_{in}(k) + B \cdot P_{heat}(k) + E \cdot T_{out}(k)$$

Máš k dispozici tabulku, kde v každém řádku znáš všechny tyto hodnoty. Neznámé jsou jen A, B, E. Abychom je vypočítali, potřebujeme ideálně stavy, kdy se mění vždy jen jedna věc.

Krok 1: Výpočet koeficientu B (Vliv topení)
Najdi v datech úsek (např. v noci), kdy byla **venkovní teplota konstantní** a **vnitřní teplota se ustálila**, a pak jsi zapnul topení na fixní výkon.
Pokud se teplota v hale zvedla z **20 °C** na **21 °C** za jednu hodinu při výkonu topení **100 kW**, můžeme zjednodušeně říct:

$$B \approx \frac{T_{in}(k+1) - T_{in}(k)}{P_{heat}(k)}$$

(Předpokládáme, že vliv venkovní teploty byl v tu chvíli nulový nebo vyrovnaný).

Krok 2: Výpočet koeficientu E (Vliv venkovní teploty)

Najdi úsek, kdy bylo **topení vypnuté** ($P_{heat} = 0$). Pak se rovnice zjednoduší na:

$$T_{in}(k+1) = A \cdot T_{in}(k) + E \cdot T_{out}(k)$$

A protože víme z předchozího vysvětlení, že $A = 1 - E$, můžeme dosadit:

$$T_{in}(k+1) = (1-E) \cdot T_{in}(k) + E \cdot T_{out}(k)$$

Z toho úpravou dostaneme:

$$E = \frac{T_{in}(k+1) - T_{in}(k)}{T_{out}(k) - T_{in}(k)}$$

Příklad: Pokud je venku $0\ ^\circ C$, uvnitř $20\ ^\circ C$ a za hodinu teplota klesne na $19\ ^\circ C$:

$$E = \frac{19-20}{0-20} = \frac{-1}{-20} = 0,05$$

Krok 3: Dopočet koeficientu A (Setrvačnost systému)
Jakmile máš **E**, koeficient **A** je prostý zbytek do jedničky (v modelu bez ztrát energie jinam):

$$A = 1 - E = 1 - 0,05 = 0,95$$

9. Jak to udělat z tabulky přesněji (Lineární regrese)

Ruční výpočet z jednoho řádku je náchylný k chybám (náhodný závan větru, někdo otevřel dveře). Pokud chceš být přesnější, použij v Excelu/LibreOffice Calc funkci **LINREGRESE**.

1. Vytvoř si sloupec **Y**, což bude rozdíl teplot: $T_{in}(k+1) - T_{in}(k)$.
2. Vytvoř si sloupce **X1, X2, X3**:
   - **X1:** aktuální vnitřní teplota $T_{in}(k)$
   - **X2:** výkon topení $P_{heat}(k)$
   - **X3:** venkovní teplota $T_{out}(k)$
3. Použij funkci lineární regrese na tyto sloupce. Koeficienty, které ti vyjdou, budou tvá A, B, E.

Na co si dát pozor:

- **Časový krok (Δt):** Musíš být konzistentní. Pokud počítáš s daty po hodinách, výsledné koeficienty platí jen pro hodinový krok.
- **Otevřené okno:** Pokud máš v datech záznam o otevřeném okně, tyto řádky při výpočtu základních A, B, E **vynechej**. Okno ti drasticky změní R (izolaci), takže by ti "rozhasilo" průměrné hodnoty haly. Pro okno si pak můžeš spočítat samostatnou sadu koeficientů (tzv. poruchový model).

10. Postupné zpřesňování konstant „samoučením“.

V praxi se používají dva hlavní přístupy: buď se model jednou za čas "přeučí" z celé historie, nebo se používá algoritmus, který se učí za pochodu s každým novým řádkem dat.

Zde je vysvětlení, jak to funguje technicky i logicky:

a/ Dávkové zpřesňování (Batch Learning)
Představ si, že máš tabulku v Excelu. **První den** máš 24 řádků (každou hodinu jeden). Spočítáš regresi a dostaneš nějaké koeficienty **A, B, E**.
Druhý den: Máš už 48 řádků. Spustíš regresi znovu na **celou** tabulku.
Týden: Máš 168 řádků.

Proč se to zpřesňuje?
Statistika funguje na principu potlačení šumu. Jeden den mohlo být větrno, někdo mohl nechat otevřená vrata nebo v hale běžel mimořádně silný stroj. V malém vzorku dat (24 hodin) tyhle "anomálie" koeficienty zkreslí. Ve velkém vzorku (např. měsíc dat) se tyto náhodné vlivy vzájemně vyruší a model začne vidět tu "pravou" fyzikální podstatu budovy.

b/ Rekurzivní identifikace (RLS – Recursive Least Squares)
Tohle je "chytrý" způsob, který se často používá v řídicích systémech. Algoritmus neukládá tisíce řádků, ale má v paměti aktuální hodnoty **A, B, E** a tzv. **matici kovariance** (míru nejistoty).

Jakmile dorazí nová data z poslední hodiny:
1. **Predikce:** Model zkusí odhadnout, jaká bude teplota teď, podle starých koeficientů.
2. **Chyba:** Změří se rozdíl mezi odhadem a skutečností (např. model čekal 20.5 °C, ale je tam 20.8 °C).
3. **Korekce:** Algoritmus "cukne" s koeficienty **A, B, E** tak, aby příště udělal menší chybu.

c/ Zapomínání (Forgetting Factor) – Klíč k realitě
Tohle je v průmyslu nejdůležitější. Budova se v čase mění. V létě jsou jiné podmínky než v zimě, izolace může navlhnout, nebo se v hale změní technologie.

Aby se model nezpřesňoval k "průměru za posledních 5 let", používá se tzv. **koeficient zapomínání**. Stará data mají v regresi menší váhu než ta nová.
* Model se "učí" neustále.
* Pokud se v hale postaví nová příčka, model si toho během pár dní všimne, protože mu přestanou vycházet predikce, a koeficient **A** (setrvačnost) se sám automaticky upraví na novou hodnotu.

11. Postup při „samoučení“

Pokud to budeš programovat nebo počítat, tvůj "zpřesňovací cyklus" bude vypadat takto:

1. **Start:** Použiješ odhadnuté koeficienty **A, B, E** z prvních 48 hodin dat.
2. **Provoz:** Každou hodinu uložíš reálná data do databáze.
3. **Adaptace:** Jednou týdně (např. v neděli v noci) spustíš skript, který vezme data za poslední měsíc a přepočítá (zpřesní) koeficienty **A, B, E** pro další týden.

Proč je to lepší než fixní model?
Kdybys parametry **R** a **C** vypočítal jen jednou z projektové dokumentace haly, nikdy bys netrefil realitu. Projektant počítá s ideálními materiály. Tvůj model "z tabulky" ale vidí, že tam jsou netěsnosti u oken, že tam jsou tepelné mosty a že stroje v hale vyzařují teplo. Model se prostě "naučí" pravdu o té konkrétní hale, i když mu o ní nic neřekneš.

12. Kód pro výpočet koeficientů (Identifikace systému)

Pro zpřesnění koeficientů použijeme lineární regresi z knihovny `**scikit-learn**`. Tento kód v podstatě říká: „Tady máš historii toho, co se dělo, a najdi mi taková čísla **A, B, E**, aby rovnice co nejlépe seděla na realitu.“

```python
from sklearn.linear_model import LinearRegression
import numpy as np

def update_model_parameters(history_data):
  """
  history_data: Tabulka (NumPy array nebo Pandas DataFrame) obsahující:
  [T_in_aktualni, T_in_nasledujici, P_vykon, T_venkovni]
  """
  # Příprava "vstupů" (X) a "výstupu" (y) pro regresi
  # Rovnice: T_next = A*T_in + B*P + E*T_out
  X = history_data[:, [0, 2, 3]] # Sloupce: T_in(k), P(k), T_out(k)
  y = history_data[:, 1]         # Sloupec: T_in(k+1)

  # Vytvoření a natrénování modelu
  # fit_intercept=False, protože chceme koeficienty přímo pro naši rovnici
  model = LinearRegression(fit_intercept=False)
  model.fit(X, y)

  # Získání koeficientů
  A, B, E = model.coef_

  return A, B, E
```

13. Celkové řešení: Propojení učení a optimalizace

Tento skript spojuje vše dohromady. Nejdříve se "učí" z historie a pak vypočítá ideální plán topení na 12 hodin dopředu.

```python
import numpy as np
from scipy.optimize import minimize
from sklearn.linear_model import LinearRegression

class SmartHeatingSystem:
  def __init__(self):
    # Výchozí odhady (pokud nemáme žádná data)
    self.A, self.B, self.E = 0.95, 0.01, 0.05

  def learn_from_history(self, history):
    """Kousek kódu, který aktualizuje koeficienty z tvé tabulky"""
    X = history[:, [0, 2, 3]] # T_in, P_heat, T_out
    y = history[:, 1]         # T_in_next
    reg = LinearRegression(fit_intercept=False).fit(X, y)
    self.A, self.B, self.E = reg.coef_
    print(f"Model aktualizován: A={self.A:.4f}, B={self.B:.4f}, E={self.E:.4f}")

  def predict_next_temp(self, T_in, P_heat, T_out):
    """Matematické jádro (stavový prostor)"""
    return self.A * T_in + self.B * P_heat + self.E * T_out

  def objective(self, u, T_start, T_out_forecast, schedule, T_setpoint):
    """Cílová funkce pro scipy.optimize (minimalizace nákladů)"""
    cost = 0
    T_curr = T_start
    for i in range(len(u)):
      # 1. Výpočet budoucí teploty podle naučených koeficientů
      T_next = self.predict_next_temp(T_curr, u[i], T_out_forecast[i])

      # 2. Penalizace za spotřebovanou energii (P_heat)
      cost += u[i]

      # 3. Penalizace za nedodržení komfortu (pouze v provozní době)
      if schedule[i] == 1 and T_next < T_setpoint:
        cost += (T_setpoint - T_next) * 100 # Váha pokuty za zimu

      T_curr = T_next
    return cost

# --- SIMULACE PROVOZU ---
```

```
# 1. Představme si nasbíraná data z tabulky (5 hodin provozu)
# Formát: [T_in(k), T_in(k+1), P_heat(k), T_out(k)]
data_z_minulosti = np.array([
   [20.0, 20.2, 50, 5],
   [20.2, 20.5, 60, 4],
   [20.5, 20.3, 10, 2],
   [20.3, 19.8, 0, 0],
   [19.8, 19.5, 0, -2]
])

system = SmartHeatingSystem()

# 2. AKTUALIZACE: Systém se naučí vlastnosti haly z dat
system.learn_from_history(data_z_minulosti)

# 3. OPTIMALIZACE: Naplánujeme topení na příštích 12 hodin
T_venku_predpoved = [ -2, -3, -3, -2, 0, 2, 4, 5, 6, 6, 5, 4 ]
provozni_doba =     [  0,  0,  0,  0, 0, 1, 1, 1, 1, 1, 1, 1 ] # Start v 6:00
aktualni_teplota = 19.5
zadana_teplota = 21.0

res = minimize(
   system.objective,
   x0=np.zeros(12), # Počáteční odhad výkonu (vše vypnuto)
   args=(aktualni_teplota, T_venku_predpoved, provozni_doba, zadana_teplota),
   bounds=[(0, 100)] * 12 # Výkon zdroje 0 až 100%
)

print("Optimální plán výkonu VZT po hodinách:")
print(np.round(res.x, 1))
```

Výhody takového způsobu výpočtu

1. **Samo-oprava:** Pokud do datové tabulky přidáš řádky, kde bylo otevřené okno (a v datech o tom máš příznak), můžeš buď tyto řádky ignorovat, nebo do regrese přidat čtvrtý koeficient (poruchu).
2. **Scipy Optimize:** Tato knihovna "nevidí" do tvých rovnic. Ona prostě zkouší měnit výkon topení v čase a sleduje, co to udělá s výslednou cenou (`cost`). Díky tomu, že zná tvé **A, B, E**, dokáže zjistit, že je levnější začít topit mírně už ve 3:00 ráno, než to "osolit" na 100 % v 6:00.
3. **Flexibilita:** Pokud se změní cena elektřiny (např. vysoký/nízký tarif), prostě jen v tom cyklu `for` vynásobíš `u[i]` aktuální cenou. Model se okamžitě přizpůsobí a bude topit v době, kdy je to nejlevnější.

Hodnoty v sekci `__init__` slouží jako výchozí bod (seed):

```
    # Výchozí odhady (pokud nemáme žádná data)
     self.A, self.B, self.E = 0.95, 0.01, 0.05
```

V praxi to funguje následovně:

1. **OFF-line fáze:** Vezmeš historická data, proženeš je tím regresním skriptem (identifikací) a získáš konkrétní čísla pro svou halu (např. A=0.982, B=0.005, $E=0.018).
2. **Nasazení:** Tato čísla vložíš do `self.A, self.B, self.E`. Systém tak začne pracovat s modelem, který už „zná“ realitu, a nemusí začínat od nuly.
3. **ON-line fáze:** Jakmile systém běží, můžeš metodu `learn_from_history` volat periodicky (např. jednou týdně) s nejnovějšími daty, aby se model dál jemně ladil.

Proč je to dobrý nápad?

Bezpečnost při startu: Pokud bys tam nechal náhodná čísla, první predikce a plány topení by mohly být úplně mimo (buď přetopeno, nebo zima). S OFF-line daty začínáš "na jistotu".
Fyzikální smysl: Tvůj OFF-line výpočet ti potvrdí, že model dává smysl. Například součet **A + E** by se měl blížit hodnotě **1.0**. Pokud by se blížil nule, víš, že je v datech chyba, ještě než to pustíš do ostrého provozu.

Tip pro implementaci
Pokud chceš být precizní, můžeš si ty OFF-line získané konstanty uložit do konfiguračního souboru (např. `config.json` nebo `.env`), aby ses nemusel vrtat přímo ve zdrojovém kódu pokaždé, když zjistíš přesnější parametry.

```python
# Příklad, jak by vypadal začátek tvé třídy s reálnými daty:
class SmartHeatingSystem:
  def __init__(self):
    # Hodnoty získané z OFF-line analýzy dat za leden-březen
    self.A = 0.9852  # Hala drží teplo velmi dobře
    self.B = 0.0041  # Výkon zdroje zvedá teplotu pomalu (velká hala)
    self.E = 0.0148  # Vliv venkovní teploty je mírný díky izolaci
```

Tímto způsobem zajistíš, že tvůj MPC (Model Predictive Control) bude mít hned od první sekundy po spuštění velmi přesný odhad, jak se hala bude chovat.

14. Celkové řešení učení a optimalizace doplněné o komentáře

```python
# --- 1. IMPORT KNIHOVEN (NÁSTROJŮ) ---
# Knihovny jsou balíčky hotových funkcí, abychom nemuseli vše vymýšlet od nuly.

import numpy as np          # Pro pokročilé matematické operace a práci s poli čísel.
import pandas as pd         # Pro snadnou práci s tabulkami (podobně jako v Excelu).
from scipy.optimize import minimize  # Speciální "hledač" nejlepšího řešení
(optimalizátor).
from sklearn.linear_model import LinearRegression # Nástroj pro hledání vztahů v datech
(regrese).

# --- 2. DEFINICE HLAVNÍHO OBJEKTU (SYSTÉMU) ---
# Třída (class) je jako "krabice", která v sobě drží data i funkce pro ovládání haly.

class InteligentniVzt:
  def __init__(self):
    """
    Tato funkce se spustí hned při vytvoření systému.
    Nastaví základní parametry, pokud ještě nemáme žádná data.
    """
    # A = setrvačnost, B = síla topení, E = vliv venkovní teploty
    self.A, self.B, self.E = 0.98, 0.005, 0.015

  def nacti_data_a_nauc_se(self, soubor_csv):
    """
    OFF-LINE ČÁST: Tato funkce otevře tabulku s historií a
    vypočítá (naučí se) přesné vlastnosti tvé haly.
    """
    try:
      # Načteme soubor CSV pomocí knihovny pandas
      df = pd.read_csv(soubor_csv)
      # Převedeme tabulku na čistá čísla (pole)
      data = df.values

      # X jsou "příčiny": (vnitřní teplota, výkon topení, venkovní teplota)
      X = data[:, [0, 2, 3]]
      # y je "následek": (vnitřní teplota v příští hodině)
      y = data[:, 1]

      # Vytvoříme stroj na lineární regresi
      regrerse = LinearRegression(fit_intercept=False)
      # Spustíme proces učení (hledání nejlepších koeficientů A, B, E)
      regrerse.fit(X, y)
```

```
        # Uložíme výsledky do paměti našeho systému
        self.A, self.B, self.E = regrerse.coef_

        print(f"--- Fáze učení dokončena ---")
        print(f"Parametry haly -> A: {self.A:.4f}, B: {self.B:.4f}, E: {self.E:.4f}")

    except Exception as e:
        # Pokud soubor neexistuje nebo je v něm chyba, program nespadne
        print(f"Pozor: Data se nepodařilo načíst ({e}). Jedu podle odhadu.")

def model_teploty(self, T_in, P, T_out):
    """
    Základní matematická rovnice (stavový prostor).
    Vypočítá, jaká bude teplota za hodinu, když známe aktuální stav.
    """
    return self.A * T_in + self.B * P + self.E * T_out

def cilova_funkce(self, u, T_start, T_venku_predpoved, program, T_pozadovana):
    """
    Tohle je "mozek" optimalizace. Hodnotí, jak dobrý je plán topení 'u'.
    Vrací 'cenu' (čím nižší číslo, tím lepší plán).
    """
    naklady = 0
    T_aktualni = T_start

    # Procházíme plán hodinu po hodině (cyklus for)
    for i in range(len(u)):
        # 1. Odhadneme teplotu v příští hodině pomocí našeho naučeného modelu
        T_dalsi = self.model_teploty(T_aktualni, u[i], T_venku_predpoved[i])

        # 2. Přičteme spotřebu energie (u[i] je výkon v dané hodině)
        naklady += u[i]

        # 3. Kontrola komfortu: Pokud je v plánu '1' (provoz), musí tam být teplo.
        # Pokud je tam zima, přičteme k nákladům obrovskou pokutu (1000).
        if program[i] == 1 and T_dalsi < T_pozadovana:
            # Rozdíl teplot vynásobíme vysokým číslem, aby se tomu algoritmus vyhnul
            naklady += (T_pozadovana - T_dalsi) * 1000

        # Posuneme čas: teplota, kterou jsme spočítali, je teď startovní pro další hodinu
        T_aktualni = T_dalsi

    return naklady

# --- 3. SPUŠTĚNÍ CELÉHO VÝPOČTU ---

# Vytvoříme instanci našeho systému
```

```
vzt = InteligentniVzt()

# KROK A: Načtení historie (pokud existuje historie_dat.csv, parametry A,B,E se zpřesní)
vzt.nacti_data_a_nauc_se('historie_dat.csv')

# KROK B: Aktuální podmínky (vstup od tebe)
T_venku_12h = [0, -1, -2, -2, -1, 0, 2, 3, 4, 5, 5, 4] # Předpověď počasí na 12h
program =     [0,  0,  0,  0,  0, 1, 1, 1, 1, 1, 1, 1] # Kdy má být v hale teplo (1=ano)
T_aktualni_v_hale = 19.2
T_pozadovana_v_provozu = 21.0

# KROK C: Spuštění hledače ideálního řešení (Optimalizátor)
# Funkce minimize zkouší různé kombinace výkonu 'u', dokud nenajde nejnižší 'naklady'.
vysledek = minimize(
   vzt.cilova_funkce,            # Co chceme minimalizovat
   x0=np.zeros(12),              # Počáteční tip (začni s vypnutým topením)
   args=(T_aktualni_v_hale, T_venku_12h, program, T_pozadovana_v_provozu), # Data
pro výpočet
   bounds=[(0, 100)] * 12        # Omezení: výkon každou hodinu musí být mezi 0 a 100 %
)

# KROK D: Výpis výsledků na obrazovku
print("\n--- NAVRŽENÝ PLÁN VÝKONU (0-100 %) ---")
# Cyklus enumerate nám dá číslo hodiny i vypočítaný výkon
for hodina, vykon in enumerate(vysledek.x):
   stav = "PROVOZ (CÍL 21°C)" if program[hodina] == 1 else "ÚTLUM (ŠETŘENÍ)"
   # Vypíšeme hodinu, stav a zaokrouhlený výkon na jedno desetinné místo
   print(f"Hodina {hodina+1:2d} | {stav:18s} | Výkon: {vykon:5.1f} %")
```

Vysvětlení některých programových obratů:

`try ... except`: To je taková pojistka. Program se pokusí provést kód v `try` (načíst soubor). Pokud soubor chybí, neskončí to chybovou hláškou na celou obrazovku, ale skočí to do `except` a vypíše to tvoje varování.
`for i in range(len(u))`: Standardní smyčka. Říká: „Udělej tohle tolikrát, kolik hodin má náš plán (v našem případě 12x).“
`u`: V tomto skriptu je to označení pro **výkon zdroje**. Je to zkratka z teorie řízení (u = control input).
`self`: To je v Pythonu způsob, jakým funkce uvnitř třídy sahá na své vlastní proměnné (např. aby si vzala svou hodnotu setrvačnosti `self.A`).

15. Formát vstupní tabulky a časová značka

V programování a analýze dat často platí, že to, co v tabulce **není** (explicitní čas), je definováno **strukturou** (pořadím řádků).

Tady je vysvětlení, jak to v tomto modelu funguje a jak by měla vstupní tabulka vypadat:

1. Časová značka je "skrytá" v řádku
Pro tento konkrétní matematický model (regresi) **nepotřebujeme znát přesné datum a čas** (např. 14. 5. 2024 10:00). Potřebujeme pouze vědět, že data v jednom řádku na sebe bezprostředně navazují.

Tvůj model se učí z **přechodů**. Proto v tabulce vidíš sloupce `T_in` (teď) a `T_in_next` (za hodinu).

Struktura tvého CSV by měla vypadat takto (nová data doplňuj vždy na konec):

T_in (nyní)	T_in_next (za hodinu)	P (výkon zdroje)	T_out (venku)
20.1	20.3	60	5
20.3	20.5	60	4
20.5	20.4	10	3

Doplňování: Nová data se **vždy doplňují na konec**. Čím delší je tabulka, tím více "zkušeností" model má.
Časová značka: V tomto modelu ji nepotřebujeme, protože předpokládáme, že interval mezi řádky je vždy stejný (např. 1 hodina). Pokud bys měl data jednou po 10 minutách a podruhé po hodině, model by přestal fungovat, protože koeficienty **A, B, E** jsou vázány na konkrétní délku kroku (**Delta t**).

2. Jak prakticky ukládat data?
V reálném provozu tvůj systém pravděpodobně loguje data do databáze nebo jiného CSV s časovými značkami. Pro náš Python skript ale musíš připravit "očištěnou" verzi.

Příklad, jak by vypadal tvůj pracovní postup v Xubuntu:
1. Máš hlavní log soubor, kde je: `Čas, Teplota_Vnitřní, Výkon, Teplota_Venkovní`.
2. Jednou za čas (nebo skriptem) z něj vytvoříš soubor `historie_dat.csv` tak, že vezmeš vnitřní teplotu z času **T** a dáš ji vedle vnitřní teploty z času **T+1**.

3. Co když chceš časové značky zachovat?
Pokud bys trval na tom, že v CSV chceš mít sloupec s časem (pro kontrolu), musíme skriptu říct, aby ho při výpočtu ignoroval.

Úprava skriptu pro tabulku s časem:

Pokud by tvé CSV vypadalo takto: `Cas, T_in, T_in_next, P, T_out`, stačí v kódu změnit řádek, kde vybíráme sloupce pro učení:

```python
# Místo původního:
X = data[:, [0, 2, 3]]

# Napíšeme (přeskočíme první sloupec s časem):
X = data[:, [1, 3, 4]]
y = data[:, 2]
```

Shrnutí:
1. **Nepřepisuj stará data.** Model potřebuje historii, aby mohl porovnat různé situace (když se topilo hodně vs. málo).
2. **Doplňuj na konec.** Každý nový řádek je pro model nová "lekce".
3. **Dodržuj interval.** Pokud se rozhodneš, že jeden řádek = 1 hodina, musí to platit v celém souboru.

16. Připojení regrese na předpověď počasí

Skript, který stahuje každou hodinu data z Open-Meteo funguje tak, že každou hodinu uloží do souboru buffer_pocasi.csv 25 údajů oddělených středníkem. První údaj je časová značka, pak následuje 12 údajů o teplotě v okamžiku časové značky a jedenácti následujících hodinách, pak následuje 12 údajů o oblačnosti v okamžiku časové značky a jedenácti následujících hodinách.

Teď je třeba hlavní skript naučit, jak si z toho souboru `buffer_pocasi.csv` data vytáhnout a jak do svého matematického modelu přidat tu oblačnost.

Předpokládejme, že tvoje historie v `historie_dat.csv` už také obsahuje sloupec pro oblačnost (aby se skript mohl naučit, jak moc slunce halu ohřívá).

Zde je upravený skript, který počítá s tvým novým souborem a zahrnuje vliv slunce:

```python
import numpy as np
import pandas as pd
from scipy.optimize import minimize
from sklearn.linear_model import LinearRegression

class InteligentniVzt:
  def __init__(self):
    # Přidali jsme koeficient G (Gains) pro vliv slunce/oblačnosti
    self.A, self.B, self.E, self.G = 0.98, 0.005, 0.015, 0.001

  def nacti_data_a_nauc_se(self, soubor_csv):
    """Učení z historie (přidán sloupec pro oblačnost)"""
```

```
        try:
            df = pd.read_csv(soubor_csv)
            data = df.values

            # X: T_in, P, T_out, Oblačnost (přidali jsme 4. sloupec)
            X = data[:, [0, 2, 3, 4]]
            y = data[:, 1]        # T_in_next

            reg = LinearRegression(fit_intercept=False).fit(X, y)
            self.A, self.B, self.E, self.G = reg.coef_
            print(f"Model naučen: A={self.A:.4f}, B={self.B:.4f}, E={self.E:.4f},
G={self.G:.4f}")
        except:
            print("Jedu na výchozí parametry.")

    def model_teploty(self, T_in, P, T_out, Oblak):
        """Rozšířená rovnice o vliv oblačnosti"""
        # G * Oblak představuje tepelný zisk ze slunce
        return self.A * T_in + self.B * P + self.E * T_out + self.G * Oblak

    def cilova_funkce(self, u, T_start, predpoved_T, predpoved_O, program, T_pozadovana):
        naklady = 0
        T_aktualni = T_start
        for i in range(len(u)):
            # Používáme teplotu i oblačnost pro každou hodinu
            T_dalsi = self.model_teploty(T_aktualni, u[i], predpoved_T[i], predpoved_O[i])
            naklady += u[i]
            if program[i] == 1 and T_dalsi < T_pozadovana:
                naklady += (T_pozadovana - T_dalsi) * 1000
            T_aktualni = T_dalsi
        return naklady

# --- FUNKCE PRO ČTENÍ TVÉHO BUFFERU ---

def nacti_buffer_pocasi(soubor):
    """
    Načte tvých 25 hodnot oddělených středníkem.
    Vrací seznam 12 teplot a 12 hodnot oblačnosti.
    """
    # Načtení jednoho řádku, středník jako oddělovač
    data = pd.read_csv(soubor, sep=';', header=None).values[0]

    # data[0] je časová značka (přeskočíme)
    teploty = data[1:13]   # 1. až 12. údaj (teploty)
    oblacnost = data[13:25] # 13. až 24. údaj (oblačnost)

    return teploty, oblacnost
```

```
# --- SPUŠTĚNÍ ---

vzt = InteligentniVzt()
vzt.nacti_data_a_nauc_se('historie_dat.csv')

# Načtení čerstvých dat z tvého bufferu
T_predpoved, O_predpoved = nacti_buffer_pocasi('buffer_pocasi.csv')

# Nastavení ostatních parametrů
program = [0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1]
T_v_hale_ted = 19.5 # Toto by mělo jít ideálně z čidla
T_cil = 21.0

# Výpočet
vysledek = minimize(
  vzt.cilova_funkce,
  x0=np.zeros(12),
  args=(T_v_hale_ted, T_predpoved, O_predpoved, program, T_cil),
  bounds=[(0, 100)] * 12
)

print("\nOptimální plán s využitím předpovědi z bufferu:")
for h, v in enumerate(vysledek.x):
  print(f"Hodina {h+1:2d}: Výkon {v:5.1f} % (Venku: {T_predpoved[h]}°C, Oblačnost:
{O_predpoved[h]}%)")
```

Co se v kódu změnilo a proč:

1. **`pd.read_csv(..., sep=';', header=None)`**: Upravili jsme čtení, aby rozumělo oddělovači středníku a faktu, že v souboru nejsou názvy sloupců.
2. **Krájení dat (`data[1:13]`)**: Tvůj buffer má pevnou strukturu (1 + 12 + 12 hodnot). Skript přesně ví, že prvních 12 čísel po čase je teplota a zbytek je oblačnost.
3. **Koeficient `G`**: V matematickém modelu je teď nová proměnná.
   Důležité: Oblačnost v Open-Meteo je obvykle v procentech (0 = jasno, 100 = zataženo). Pokud slunce halu ohřívá, koeficient `G` bude v regresi vycházet jako **záporné číslo** (protože čím větší oblačnost, tím menší "zisk" tepla a tím nižší výsledná teplota). Většina lidí uvažuje logicky: **"Slunce mi halu ohřívá"**. Jenže tvůj vstupní údaj je **oblačnost**, nikoliv **sluneční svit**. Funguje to tedy nepřímo úměrně. Pokud tedy bude v rovnici `+ G oblačnost`, pak koeficient `G` vyjde pravděpodobně jako **záporné číslo**.
4. **Optimalizace**: Algoritmus teď vidí, že například v 10. hodině bude jasno (0 % oblačnosti), takže si může dovolit topit o něco méně, protože mu "pomůže" slunce.

Poznámka k integraci v Xubuntu:
Pokud tento hlavní skript pojmenuješ třeba **`regulator.py`**, můžeš si vytvořit jednoduchý

"spouštěč" (**bash skript**), který nejdřív spustí tvůj stahovač počasí a hned potom tento regulátor. Tím zajistíš, že data budou vždy čerstvá.

Tip pro zpřesnění (Oblačnost vs. Radiační zisky)
Pokud bys chtěl být v budoucnu ještě preciznější, Open-Meteo umí místo oblačnosti (v %) poskytovat i **přímé sluneční záření** (v jednotkách W/m^2 – označuje se to jako `shortwave_radiation`).

Pokud bys použil tento údaj:
1. 0 by znamenalo tmu (noc).
2. Vysoké číslo (např. 800) by znamenalo ostré polední slunce.
3. Koeficient `G` by pak byl **kladný**, což je pro lidský mozek o něco intuitivnější.

Ale pro začátek je tvůj systém s oblačností naprosto dostatečný a funkční. Máš teď v rukou v podstatě profesionální nástroj pro prediktivní řízení (MPC), který ti může ušetřit desítky procent nákladů na energie díky tomu, že "vidí do budoucnosti".

17. Regrese připojená na předpověď počasí s komentáři

Tady je ten úplný, finální skript se všemi tvými požadavky: načítáním z tvého `buffer_pocasi.csv`, zahrnutím oblačnosti do modelu a podrobným komentářem ke každému řádku, aby se v něm začátečník neztratil.

```python
# --- 1. IMPORT KNIHOVEN (NÁSTROJŮ) ---
import numpy as np          # Pro matematické operace (práce s poli a maticemi).
import pandas as pd         # Pro čtení a úpravu tabulek (CSV souborů).
from scipy.optimize import minimize  # "Hledač" nejlepšího (nejlevnějšího) řešení.
from sklearn.linear_model import LinearRegression # Nástroj, který se učí vlastnosti haly z historie.

# --- 2. DEFINICE HLAVNÍHO OBJEKTU (SYSTÉMU) ---
class InteligentniVzt:
  def __init__(self):
    """
    Nastavení základních parametrů haly (A, B, E, G).
    Tyto hodnoty se přepíšou, jakmile se skript naučí data z historie.
    """
    self.A = 0.98    # Setrvačnost (kolik tepla zůstane v hale z minulé hodiny).
    self.B = 0.005   # Síla topení (jak moc výkon VZT zvedne teplotu).
    self.E = 0.015   # Vliv venkovní teploty (izolace).
    self.G = -0.001  # Vliv oblačnosti (sluneční zisky – obvykle záporné číslo).

  def nacti_data_a_nauc_se(self, soubor_csv):
    """
```

```
    OFF-LINE ČÁST: Otevře historii a vypočítá přesné koeficienty haly.
    Očekává sloupce: T_in, T_in_next, P, T_out, Oblačnost.
    """
    try:
        # Načtení historie pomocí pandas
        df = pd.read_csv(soubor_csv)
        data = df.values

        # X jsou příčiny: T_in(nyní), Výkon(P), T_venku, Oblačnost
        X = data[:, [0, 2, 3, 4]]
        # y je následek: T_in(za hodinu)
        y = data[:, 1]

        # Vytvoření a natrénování matematického modelu
        regrese = LinearRegression(fit_intercept=False).fit(X, y)

        # Uložení naučených koeficientů do paměti
        self.A, self.B, self.E, self.G = regrese.coef_

        print(f"--- Fáze učení dokončena ---")
        print(f"Koeficienty -> A:{self.A:.4f}, B:{self.B:.4f}, E:{self.E:.4f}, G:{self.G:.4f}")
    except Exception as e:
        print(f"Učení neproběhlo ({e}). Používám výchozí nastavení.")

def model_teploty(self, T_in, P, T_out, Oblak):
    """
    Jádro výpočtu: Předpoví teplotu za hodinu podle aktuálního stavu.
    Rovnice: T_příští = A*T_nyní + B*Výkon + E*T_venku + G*Oblačnost.
    """
    return self.A * T_in + self.B * P + self.E * T_out + self.G * Oblak

def cilova_funkce(self, u, T_start, predpoved_T, predpoved_O, program, T_pozadovana):
    """
    Hledání nejlevnějšího plánu. Vrací 'cenu' (spotřebu + pokuty za zimu).
    'u' je seznam 12 hodnot výkonu, které zkouší optimalizátor.
    """
    naklady = 0
    T_aktualni = T_start

    # Procházíme budoucích 12 hodin (cyklus for)
    for i in range(len(u)):
        # Odhadneme teplotu pro konkrétní hodinu 'i'
        T_dalsi = self.model_teploty(T_aktualni, u[i], predpoved_T[i], predpoved_O[i])

        # Přičteme výkon do nákladů (chceme topit co nejméně)
        naklady += u[i]
```

```
        # Pokud má být hala v provozu a je v ní zima, přičteme velkou pokutu
        if program[i] == 1 and T_dalsi < T_pozadovana:
            naklady += (T_pozadovana - T_dalsi) * 1000

        # Teplota z konce této hodiny je startovní pro tu další
        T_aktualni = T_dalsi

    return naklady

# --- 3. POMOCNÁ FUNKCE PRO TVŮJ BUFFER POČASÍ ---

def nacti_buffer_pocasi(soubor):
    """
    Načte tvůj soubor se středníky.
    Přeskočí čas a rozdělí data na 12 teplot a 12 hodnot oblačnosti.
    """
    # Načtení řádku: sep=';' značí středník, header=None znamená, že tam nejsou názvy
sloupců
    data = pd.read_csv(soubor, sep=';', header=None).values[0]

    # Rozkrájení dat podle tvého formátu (index 0 je čas)
    teploty = data[1:13]    # Hodnoty na pozici 1 až 12
    oblacnost = data[13:25] # Hodnoty na pozici 13 až 24

    return teploty, oblacnost

# --- 4. SPUŠTĚNÍ CELÉHO PROCESU ---

# Vytvoření instance regulátoru
vzt = InteligentniVzt()

# KROK A: Učení se z historie (OFF-line část)
vzt.nacti_data_a_nauc_se('historie_dat.csv')

# KROK B: Načtení aktuální předpovědi z tvého bufferu (ON-line část)
try:
    T_predpoved, O_predpoved = nacti_buffer_pocasi('buffer_pocasi.csv')
except:
    print("Chyba: Nepodařilo se načíst buffer_pocasi.csv!")
    # Náhradní data pro případ výpadku bufferu
    T_predpoved = [0]*12
    O_predpoved = [50]*12

# KROK C: Aktuální stav z haly a program
T_v_hale_nyni = 19.5          # Zde by měl být údaj z tvého teploměru
T_pozadovana = 21.0           # Teplota, kterou chceš mít v provozní době
provozni_program = [0,0,0,0,0,1,1,1,1,1,1,1] # Kdy je hala v provozu (1 = ano)
```

```
# KROK D: Spuštění optimalizátoru (Hledání nejlepšího 'u')
vysledek = minimize(
   vzt.cilova_funkce,        # Naše funkce pro hodnocení nákladů
   x0=np.zeros(12),          # Začínáme s odhadem "všechno vypnuto"
   args=(T_v_hale_nyni, T_predpoved, O_predpoved, provozni_program, T_pozadovana), #
Data
   bounds=[(0, 100)] * 12     # Výkon musí být v rozmezí 0 až 100 %
)

# KROK E: Výpis výsledného plánu na obrazovku
print("\n--- NAVRŽENÝ PLÁN VÝKONU ---")
for h, vykon in enumerate(vysledek.x):
   # h je hodina (0-11), vykon je vypočítané %
   print(f"Hodina {h+1:2d} | Venku: {T_predpoved[h]:4.1f}°C | Oblačnost:
{O_predpoved[h]:3.0f}% | Výkon VZT: {vykon:5.1f} %")
```

Teď je to kompletní "balíček". Skript si sám sáhne do historie, aby se naučil, jak hala reaguje, pak si sáhne do bufferu pro čerstvé počasí a nakonec vyhodí ideální plán topení. Stačí jen v Xubuntu spustit.

Kdybys v budoucnu narazil na to, že chceš model rozšířit třeba o ty ceny elektřiny (aby to topilo víc v levném tarifu) nebo o detekci otevřených oken, stačí se ozvat.